

J1 — Introduction à R et à la pensée spatiale pour démographes

Code

Atelier IFORD × GDSG · Yaoundé, 27 juillet 2026

AUTEUR·RICE
Ramesesse Dzita (GDSG)

DATE DE PUBLICATION
27 juillet 2026

1 Objectifs et public cible

Ce premier jour vise à **mettre tout le monde au même niveau** sur R, en l'appliquant immédiatement à des questions démographiques camerounaises. Le public — statisticiens d'INS, démographes IFORD, techniciens de ministères, ONG — manipule déjà Excel et SPSS / Stata ; l'enjeu est de leur faire **basculer leur intuition tableur vers la grammaire R**, et de leur faire toucher du doigt qu'**un tableau peut avoir une géométrie**.

À la fin du jour, chaque participant doit pouvoir :

1. Naviguer dans RStudio et créer un projet reproductible avec [here](#) .
2. Manipuler un `data.frame` issu d'une enquête démographique camerounaise (vecteurs, indexation, filtres, agrégats).
3. Comprendre ce qui distingue un `data.frame` d'un objet spatial `sf` .
4. Charger les limites administratives du Cameroun et produire une première carte propre.
5. Énoncer ce qu'est un **système de coordonnées de référence** (CRS) et pourquoi le confondre cause des erreurs de plusieurs kilomètres.

Les exercices pratiques de la journée sont rassemblés dans [exercice.qmd](#) (énoncés) et [corrige.qmd](#) (solutions commentées). La démo ci-dessous ne contient **que** la présentation linéaire ; gardez les deux autres fichiers ouverts en parallèle.

2 Pré-requis et environnement

Avant d'avancer, on s'assure que R, Quarto et les packages requis sont prêts. Le bloc suivant interroge la version de R, tente d'attraper la version de Quarto via `system2()` (sans planter si Quarto n'est pas sur le PATH grâce au `tryCatch()`), puis vérifie pour chaque package du jour s'il est installé en croisant la liste attendue avec `installed.packages()` . La sortie est un tableau `package / installed` à scanner d'un coup d'œil.

```
R.version.string
```

```
[1] "R version 4.6.0 (2026-04-24 ucrt)"
```

```
quarto_version <- tryCatch(system2("quarto", "--version", stdout = TRUE),  
                           error = function(e) NA)  
quarto_version
```

```
[1] "1.9.38"
```

```
# Packages installés ?
pkgs_jour1 <- c("here", "fs", "tidyverse", "janitor", "skimr",
               "haven", "readxl", "sf", "tmap", "ggplot2", "rnaturalearth")
installed <- pkgs_jour1 %in% rownames(installed.packages())
data.frame(package = pkgs_jour1, installed = installed)
```

package	installed
<chr>	<lgl>
here	TRUE
fs	TRUE
tidyverse	TRUE
janitor	TRUE
skimr	TRUE
haven	TRUE
readxl	TRUE
sf	TRUE
tmap	TRUE
ggplot2	TRUE

1-10 of 11 rows

Previous **1** [2](#) [Next](#)

Si un package manque, on l'installe **maintenant**, pas en fin de séance. Le bloc ci-dessous est marqué `#| eval: false` : Quarto le rend pour la lecture mais ne l'exécute pas (sinon il tenterait l'installation à chaque rendu, ce qu'on ne veut surtout pas).

```
install.packages(pkgs_jour1[!installed], dependencies = TRUE)
```

3 Première session — R comme calculatrice statistique

 **~1h30**. Ralentir sur les types et l'indexation, c'est là que les vrais débutants accrochent.

3.1 Pourquoi R quand on a Stata, SPSS ou Excel ?

Excel est un outil de saisie et de visualisation rapide, pas un outil d'analyse reproductible. Stata et SPSS sont d'excellents outils d'analyse mais leurs licences sont coûteuses et leur écosystème géospatial est pauvre. R est :

- **Gratuit et open source**, donc déployable dans toute institution africaine sans contrainte budgétaire.
- **Scriptable**, donc reproductible (chaque étape est tracée).
- **Riche en démographie et en géospatial** : `sf`, `terra`, `tmap`, `spdep`, `rdhs`, `wopr`, `INLA`.
- **Universellement adopté** par WorldPop, le DHS Program, l'INED, l'INSEE, l'INSPQ.

Stata reste un excellent outil — l'objectif ici n'est pas de le remplacer mais de l'**augmenter** d'une dimension spatiale.

3.2 Vecteurs : la brique élémentaire

Un **vecteur** est la structure de base de R : une séquence ordonnée d'éléments du **même type**. On en crée un avec `c()` (« combine »). Le bloc qui suit construit deux vecteurs parallèles — un **character** (les 10 régions du Cameroun) et un **numeric** (population estimée) — puis interroge leurs propriétés : `length()` retourne le nombre d'éléments, `sum()` la somme, `mean()` la moyenne, `range()` le couple (min, max). Chaque appel produit un seul nombre (ou un couple).

```
# Un vecteur numérique : les 10 régions du Cameroun par population
# (estimations BUCREP 2019, ordres de grandeur, à actualiser avec RGPH 4)
regions <- c("Adamaoua", "Centre", "Est", "Extrême-Nord", "Littoral",
             "Nord", "Nord-Ouest", "Ouest", "Sud", "Sud-Ouest")
pop_est <- c(1340000, 4400000, 870000, 4200000, 3400000,
            2700000, 2000000, 2000000, 800000, 1700000)

length(regions)
```

```
[1] 10
```

```
sum(pop_est)
```

```
[1] 23410000
```

```
mean(pop_est)
```

```
[1] 2341000
```

```
range(pop_est)
```

```
[1] 800000 4400000
```

R **vectorise** par défaut : une opération arithmétique sur un vecteur s'applique élément par élément, sans boucle. Le bloc ci-dessous divise les 10 populations par un million d'un seul coup, puis arrondit le résultat à deux décimales avec `round()`.

```
pop_millions <- pop_est / 1e6
round(pop_millions, 2)
```

```
[1] 1.34 4.40 0.87 4.20 3.40 2.70 2.00 2.00 0.80 1.70
```

Le **recyclage** est une source d'erreurs subtiles — on en parlera demain. Pour aujourd'hui : si on additionne deux vecteurs de tailles différentes, R recycle silencieusement le plus court.

3.3 Types et coercion

R distingue plusieurs types primitifs (`character`, `numeric`, `logical`, `integer` ...). Le bloc suivant les interroge avec `class()` (interprétation de haut niveau) et `typeof()` (type de stockage interne — `double` est la précision en virgule flottante 64 bits). Quand on mélange volontairement un caractère et des nombres dans `c(1, 2, "trois")`, R applique la **coercion implicite** : tout devient `character`, parce que c'est le type qui peut tout contenir. C'est la racine d'une grande famille de bugs en démographie.

```
class(regions) # character
```

```
[1] "character"
```

```
class(pop_est) # numeric
```

```
[1] "numeric"
```

```
typeof(pop_est) # double
```

```
[1] "double"
```

```
# Coercion implicite : un seul caractère contamine tout
mix <- c(1, 2, "trois")
mix
```

```
[1] "1"      "2"      "trois"
```

```
class(mix)
```

```
[1] "character"
```

En pratique démographique, on lit souvent des CSV où un code région est mal interprété comme nombre (perte des zéros en tête). On verra `readr::read_csv()` qui devine plus prudemment.

3.4 data.frame et tibble : la table de données

Le tableau est la structure dominante en démographie. R en a deux variantes principales : le `data.frame` (base R) et le `tibble` (tidyverse).

On assemble un `data.frame` avec une liste de colonnes nommées de **même longueur**. Le paramètre `stringsAsFactors = FALSE` désactive un comportement historique de R qui convertissait automatiquement les chaînes en facteurs (`factor`) — c'était une source de bugs et le défaut a changé en R 4.0, mais l'écrire explicitement reste une bonne pratique défensive. Ensuite, `as_tibble()` convertit ce `data.frame` en `tibble`, qui s'affiche de façon plus lisible et adopte des règles plus strictes.

```
df_cmr <- data.frame(
  region    = regions,
  pop_2019  = pop_est,
  capitale  = c("Ngaoundéré", "Yaoundé", "Bertoua", "Maroua", "Douala",
               "Garoua", "Bamenda", "Bafoussam", "Ebolowa", "Buea"),
  stringsAsFactors = FALSE
)
df_cmr
```

region	pop_2019	capitale
<chr>	<dbl>	<chr>
Adamaoua	1340000	Ngaoundéré

region	pop_2019	capitale
<chr>	<dbl>	<chr>
Centre	4400000	Yaoundé
Est	870000	Bertoua
Extrême-Nord	4200000	Maroua
Littoral	3400000	Douala
Nord	2700000	Garoua
Nord-Ouest	2000000	Bamenda
Ouest	2000000	Bafoussam
Sud	800000	Ebolowa
Sud-Ouest	1700000	Buea

1-10 of 10 rows

```
# Mêmes données en tibble
library(tibble)
tbl_cmr <- as_tibble(df_cmr)
tbl_cmr
```

region	pop_2019	capitale
<chr>	<dbl>	<chr>
Adamaoua	1340000	Ngaoundéré
Centre	4400000	Yaoundé
Est	870000	Bertoua
Extrême-Nord	4200000	Maroua
Littoral	3400000	Douala
Nord	2700000	Garoua
Nord-Ouest	2000000	Bamenda
Ouest	2000000	Bafoussam
Sud	800000	Ebolowa
Sud-Ouest	1700000	Buea

1-10 of 10 rows

Le **tibble** est plus strict (pas de conversion silencieuse des chaînes en facteurs, affichage tronqué, refus d'index partiel ambigu) — il sera l'objet de référence du tidyverse.

3.5 Indexation : trois conventions à connaître

R offre trois syntaxes pour accéder à une partie d'un tableau, et il faut savoir les lire toutes les trois car les forums et le code existant les mélangent. La notation `[ligne, colonne]` est la plus générale (laisser un côté vide signifie « tout »). La notation `$nom` est un raccourci pour extraire une colonne par son nom — résultat : un vecteur. La notation `[[ ]]` extrait un élément unique d'une liste/data.frame — utile quand le nom de colonne est dans une variable.

```
# Notation [ligne, colonne ]
df_cmr[1, ]          # première région
```

	region	pop_2019	capitale
	<chr>	<dbl>	<chr>
1	Adamaoua	1340000	Ngaoundéré
1 row			

```
df_cmr[, "region"]      # colonne region (vecteur)
```

```
[1] "Adamaoua"    "Centre"      "Est"         "Extrême-Nord" "Littoral"
[6] "Nord"        "Nord-Ouest"  "Ouest"       "Sud"          "Sud-Ouest"
```

```
df_cmr[df_cmr$pop_2019 > 3e6, ] # régions de plus de 3 millions
```

	region	pop_2019	capitale
	<chr>	<dbl>	<chr>
2	Centre	4400000	Yaoundé
4	Extrême-Nord	4200000	Maroua
5	Littoral	3400000	Douala
3 rows			

```
# Notation $
df_cmr$pop_2019
```

```
[1] 1340000 4400000 870000 4200000 3400000 2700000 2000000 2000000 800000
[10] 1700000
```

```
# Notation [[ ]] (élément unique)
df_cmr[["region"]]
```

```
[1] "Adamaoua"    "Centre"      "Est"         "Extrême-Nord" "Littoral"
[6] "Nord"        "Nord-Ouest"  "Ouest"       "Sud"          "Sud-Ouest"
```

En tidyverse on préfère un enchaînement de verbes. Le bloc qui suit lit comme une phrase : « prendre `tbl_cmr`, **puis** ne garder que les régions de plus de 3 millions, **puis** trier par population décroissante, **puis** garder ces trois colonnes ». Chaque verbe (`filter`, `arrange`, `select`) prend un tibble en entrée et retourne un tibble en sortie — c'est ce qui rend le pipe lisible.

```
library(dplyr)
tbl_cmr |>
  filter(pop_2019 > 3e6) |>
  arrange(desc(pop_2019)) |>
  select(region, capitale, pop_2019)
```

region	capitale	pop_2019
<chr>	<chr>	<dbl>
Centre	Yaoundé	4400000
Extrême-Nord	Maroua	4200000

region	capitale	pop_2019
<chr>	<chr>	<dbl>
Littoral	Douala	3400000

3 rows

Le **pipe natif** `|>` (R ≥ 4.1) se lit « puis ». Il rend les pipelines lisibles à voix haute et évite les imbrications du type `select(arrange(filter(...)))` qui se lisent de l'intérieur vers l'extérieur.

3.6 Le verbe `mutate` — créer une variable

`mutate()` ajoute (ou modifie) des colonnes dans un tibble. Le bloc ci-dessous calcule la population en millions, puis crée une variable catégorielle `pop_classe` via `case_when()` — l'équivalent lisible d'un `IF / ELSE IF / ELSE`. Le `TRUE ~ "Plus de 3M"` joue le rôle de la clause par défaut.

```
tbl_cmr <- tbl_cmr |>
  mutate(
    pop_millions = pop_2019 / 1e6,
    pop_classe   = case_when(
      pop_2019 < 1e6 ~ "Moins de 1M",
      pop_2019 < 3e6 ~ "1 à 3M",
      TRUE         ~ "Plus de 3M"
    )
  )
tbl_cmr
```

region	pop_2019	capitale	pop_millions	pop_classe
<chr>	<dbl>	<chr>	<dbl>	<chr>
Adamaoua	1340000	Ngaoundéré	1.34	1 à 3M
Centre	4400000	Yaoundé	4.40	Plus de 3M
Est	870000	Bertoua	0.87	Moins de 1M
Extrême-Nord	4200000	Maroua	4.20	Plus de 3M
Littoral	3400000	Douala	3.40	Plus de 3M
Nord	2700000	Garoua	2.70	1 à 3M
Nord-Ouest	2000000	Bamenda	2.00	1 à 3M
Ouest	2000000	Bafoussam	2.00	1 à 3M
Sud	800000	Ebolowa	0.80	Moins de 1M
Sud-Ouest	1700000	Buea	1.70	1 à 3M

1-10 of 10 rows

3.7 Le verbe `summarise` — agréger

`group_by()` partitionne le tibble selon une (ou plusieurs) variable(s) ; `summarise()` réduit ensuite **chaque groupe à une seule ligne** en appliquant des fonctions d'agrégation. Ici on obtient une ligne par classe de population, avec le nombre de régions, la population totale et la population moyenne dans chaque classe.

```
tbl_cmr |>
  group_by(pop_classe) |>
  summarise(
    n_regions = n(),
    pop_totale = sum(pop_2019),
    pop_moyenne = mean(pop_2019)
  )
```

pop_classe	n_regions	pop_totale	pop_moyenne
<chr>	<int>	<dbl>	<dbl>
1 à 3M	5	9740000	1948000
Moins de 1M	2	1670000	835000
Plus de 3M	3	12000000	4000000

3 rows

Cette grammaire `filter` / `mutate` / `group_by` / `summarise` couvre 80 % de ce que fait un démographe sur un microfichier. Les 20 % restants sont les jointures (J5) et les pivots (lendemain).

👉 **À pratiquer en autonomie** : exercices Q1 à Q3 de `exercice.qmd` (10–15 min). Les solutions sont dans `corrige.qmd`.

4 Session « comment demander de l'aide en R »

🕒 **~10 minutes**. À insérer entre la session 1 et la session 2. Cette compétence évite 80 % des futurs blocages.

Quand vous serez bloqué pendant l'atelier (et vous le serez), vous avez **cinq réflexes** :

4.1 1. L'aide intégrée de R

Chaque fonction installée dans R a une page d'aide accessible **sans connexion internet**. `?nom` est le raccourci ; `help(nom)` est l'équivalent verbose ; `??motclé` (deux points d'interrogation) fait une recherche full-text dans toute la doc des packages installés. Le bloc est marqué `#| eval: false` parce qu'on ne veut pas ouvrir les pages d'aide au moment du rendu HTML.

```
?mean          # ouvre l'aide de la fonction mean()
help(mean)     # équivalent
?dplyr::filter  # aide spécifique au filter de dplyr
??regression    # cherche "regression" dans toute la doc installée
```

Pour chaque package, il existe aussi des **vignettes** — tutoriels longs maintenus par les auteurs du package. `vignette(package = "sf")` liste celles disponibles, `vignette("sf1")` ouvre la première.

```
vignette(package = "sf")
vignette("sf1")
```


4.2 2. Les cheatsheets RStudio

Aide-mémoire visuels d'une ou deux pages : **RStudio Help** → **Cheatsheets** (ou <https://posit.co/resources/cheatsheets/>). Indispensables pour `dplyr`, `tidyr`, `ggplot2`, `sf`. **Téléchargez ces 4 cheatsheets en début d'atelier**, gardez-les ouvertes à côté.

4.3 3. Erreur → Google → StackOverflow

Quand vous voyez une erreur R :

1. **Copier le message exact** dans Google.
2. Les premiers résultats sont presque toujours **StackOverflow** ou **RStudio Community**.
3. Lire la **réponse acceptée** (cochée en vert) ET les commentaires.

💡 Astuce : ajoutez `R` ou `R + nom-du-package` à votre recherche (ex. `R sf st_intersection error`).

4.4 4. Forums spécialisés

- <https://community.rstudio.com> — forum officiel Posit.
- <https://geocompx.org/post/> — communauté R-spatial (Lovelace, Pebesma).

4.5 5. ChatGPT, Claude et assistants IA

Très efficaces pour le code R, à **condition de** :

- Fournir le **message d'erreur exact** (pas une paraphrase).
- Fournir un **exemple minimal reproductible** (~ 10 lignes).
- **Toujours tester** le code suggéré dans votre console — l'IA hallucine parfois des fonctions inexistantes.

Limites pédagogiques

Utilisez ces outils pour **dépasser un blocage**, pas pour **éviter de comprendre**. Si vous copiez sans relire, vous apprenez à utiliser une IA — pas R. Les deux compétences sont différentes.

5 Deuxième session — Projet RStudio et reproductibilité

🕒 ~1 heure.

5.1 Pourquoi un « projet » ?

Sans projet, le répertoire de travail R dépend de comment on a ouvert RStudio. Un script qui marche sur ta machine échouera sur celle d'un collègue dès la première ligne `read.csv("C:/Users/Ramesse/...")`. Le projet RStudio fixe la racine, et le package [here](#) donne des chemins **relatifs** à cette racine.

Créer un projet (en salle) : `File → New Project → New Directory → New Project`, nommer `tp_jour1`, le placer sous `Documents/IFORD_2026/`.

5.2 La fonction `here::here()`

`here::here()` détecte la racine du projet (présence d'un fichier `.Rproj` ou `.here`) et **reconstruit** des chemins relatifs à cette racine, dans la syntaxe de chemin native de l'OS courant. Appelée sans argument, elle retourne la racine. Avec des arguments, elle assemble les morceaux : `here("data", "monfichier.csv")` devient `C:/Users/.../monfichier.csv` sous Windows et `/home/.../monfichier.csv` sous Linux, sans qu'on ait à s'en soucier.

```
library(here)
here() # racine du projet
```

```
[1] "C:/Dev/GitHub/atelier-r-spatial-iford-2026/pedagogie"
```

```
here("data", "monfichier.csv") # chemin OS-indépendant
```

```
[1] "C:/Dev/GitHub/atelier-r-spatial-iford-2026/pedagogie/data/monfichier.csv"
```

À partir de maintenant, **aucun chemin absolu** dans nos scripts. Si un participant écrit `"C:/Users/.../bureau/fichier.csv"`, c'est un bug en attente.

5.3 Lire un fichier d'enquête démographique camerounaise

Démonstration sur un extrait ménages format Stata (typique DHS / MICS / ECAM). `haven::read_dta()` lit un fichier `.dta` en préservant les labels Stata. `dim()` retourne (nlines, ncolonnes), `names()` liste les variables (on tronque à 20 ici car un microfichier DHS en a souvent ~ 1 500). Le bloc est `eval: false` parce que le fichier réel arrive après inscription DHS.

```
library(haven)
# Lecture du fichier ménages EDS-MICS Cameroun 2018
hh <- read_dta(here("datasets", "cameroun", "dhs_mics", "CMHR71FL.DTA"))
dim(hh)
names(hh)[1:20]
```

Si le fichier n'est pas disponible aujourd'hui (inscription DHS en attente), on **simule** une structure analogue. `set.seed()` fige le générateur aléatoire pour que la sortie soit reproductible. `sample(x, n, replace = TRUE, prob = ...)` tire `n` valeurs dans `x` avec remplacement, et la probabilité de chaque valeur peut être pondérée (ici par la part de population estimée). `rpois(n, lambda)` simule un effectif de ménage suivant une loi de Poisson de moyenne `lambda`. `pmax(1, ...)` garantit qu'aucun ménage n'est de taille 0.

```
set.seed(2026)
n <- 11000 # ordre de grandeur du nombre de ménages EDS-MICS 2018
hh_sim <- tibble(
  id_menage = sprintf("HH%05d", seq_len(n)),
  region    = sample(regions, n, replace = TRUE,
                    prob = pop_est / sum(pop_est)),
```

```

milieu      = sample(c("Urbain", "Rural"), n, replace = TRUE, prob = c(0.57, 0.43)),
taille_mng  = pmax(1, round(rpois(n, lambda = 5.1))),
quintile    = sample(1:5, n, replace = TRUE),
acces_eau   = sample(c("Améliorée", "Non améliorée"), n, replace = TRUE,
                     prob = c(0.72, 0.28)),
electricite = sample(c("Oui", "Non"), n, replace = TRUE,
                     prob = c(0.61, 0.39))
)
hh_sim

```

id_menage <chr>	region <chr>	milieu <chr>	taille_mng <dbl>	quintile <int>	acces_eau <chr>	
HH00001	Nord-Ouest	Urbain	11	2	Non améliorée	
HH00002	Nord	Rural	9	3	Améliorée	
HH00003	Centre	Urbain	4	5	Améliorée	
HH00004	Extrême-Nord	Rural	5	4	Améliorée	
HH00005	Nord	Rural	10	5	Améliorée	
HH00006	Centre	Rural	3	4	Améliorée	
HH00007	Littoral	Rural	6	2	Améliorée	
HH00008	Sud-Ouest	Urbain	4	3	Non améliorée	
HH00009	Extrême-Nord	Urbain	9	1	Améliorée	
HH00010	Nord	Rural	6	1	Non améliorée	

1-10 of 10,000 rows | 1-6 of 7 columns

Previous **1** [2](#) [3](#) [4](#) [5](#) [6](#) ... [1000](#) [Next](#)

Premier coup d’œil systématique : `skimr::skim()` produit en une commande un résumé par variable (type, valeurs manquantes, distribution). C’est votre **premier réflexe** sur tout nouveau microfichier — avant même d’écrire la moindre analyse.

```

library(skimr)
skim(hh_sim)

```

Data summary

Name	hh_sim
Number of rows	11000
Number of columns	7
Column type frequency:	
character	5
numeric	2
Group variables	
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
id_menage	0	1	7	7	0	11000	0
region	0	1	3	12	0	10	0
milieu	0	1	5	6	0	2	0
acces_eau	0	1	9	13	0	2	0
electricite	0	1	3	3	0	2	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
taille_mng	0	1	5.12	2.26	1	4	5	7	15	
quintile	0	1	2.98	1.42	1	2	3	4	5	

5.4 Calcul d'indicateurs démographiques de base

On enchaîne `group_by(region) |> summarise(...)` pour produire, en une seule expression, une ligne par région avec : le nombre de ménages, la taille moyenne, le pourcentage d'urbains, d'électrifiés et d'eau améliorée. `mean(milieu == "Urbain")` exploite le fait que R convertit `TRUE/FALSE` en `1/0`, donc la moyenne d'un booléen est une proportion. `arrange(desc(...))` trie en décroissant. Aucune pondération ici — la note méthodologique qui suit explique pourquoi c'est insuffisant en production.

```
# Taille moyenne des ménages par région (sans pondération - juste démo)
hh_sim |>
  group_by(region) |>
  summarise(
    n_menages      = n(),
    taille_moyenne = mean(taille_mng),
    pct_urbain     = mean(milieu == "Urbain") * 100,
    pct_electricite = mean(electricite == "Oui") * 100,
    pct_eau_amelio = mean(acces_eau == "Améliorée") * 100,
    .groups = "drop"
  ) |>
  arrange(desc(pct_electricite))
```

region	n_menages	taille_moyenne	pct_urbain	pct_electricite
<chr>	<int>	<dbl>	<dbl>	<dbl>
Sud	370	5.127027	58.10811	61.89189
Nord	1323	5.279667	59.25926	61.60242
Est	415	4.980723	54.45783	61.44578
Extrême-Nord	1985	5.100252	57.83375	61.25945
Nord-Ouest	921	5.234528	55.91748	60.69490
Centre	2068	5.119439	57.97872	60.54159
Ouest	909	4.975798	57.86579	60.28603
Littoral	1587	5.046629	55.13548	59.04222
Adamaoua	658	5.164134	53.79939	58.66261
Sud-Ouest	764	5.119110	59.42408	57.46073

Note méthodologique importante : sur une vraie EDS-MICS, ces calculs **doivent être pondérés** par le poids ménage (`hv005 / 1e6`) et tenir compte du plan de sondage stratifié à deux degrés. Voir le package `survey` et `srvyr` . On y reviendra J5.

6 Troisième session — Qu’est-ce qu’un objet spatial ?

 **~1h30**. Pédagogie clé : insister sur l’analogie « data.frame + colonne géométrie », c’est ce qui débloque les débutants.

6.1 L’intuition

Un `data.frame` classique a des **lignes** (observations) et des **colonnes** (variables). Un objet **spatial** est un `data.frame` **augmenté d’une colonne de géométries** — un point, une ligne, un polygone. Cette colonne sait comment se positionner sur la surface de la Terre.

Le format dominant en R aujourd’hui est `sf` (Simple Features) ([Pebesma 2018](#)), qui implémente la norme OGC ISO 19125. `sf` remplace l’ancien package `sp` . On utilise **toujours `sf` en 2026**.

6.2 Le package `sf` en 3 fonctions

Avant tout `sf` repose sur trois bibliothèques C/C++ — **GDAL** lit/écrit les formats (Shapefile, GeoJSON, GeoTIFF...), **GEOS** fait les opérations géométriques (intersection, union, distance), **PROJ** gère les projections. `packageVersion("sf")` renvoie la version du package R, `sf_extSoftVersion()` celle des trois bibliothèques externes — il faut savoir les retrouver en cas de bug pour reporter un environnement clair.

```
library(sf)
packageVersion("sf")
```

```
[1] '1.1.1'
```

```
sf_extSoftVersion() # versions de GDAL, GEOS, PROJ sous-jacentes
```

GEOS	GDAL	proj.4	GDAL_with_GEOS	USE_PROJ_H
"3.14.1"	"3.12.1"	"9.7.1"	"true"	"true"
PROJ				
"9.7.1"				

Ces trois bibliothèques sont les fondations de tout SIG dans le monde. R les expose via `sf` .

6.3 Charger les limites administratives du Cameroun

On charge nos données. `source()` exécute le script `fetch_data.R` qui définit la fonction `fetch_gadm_cameroon()`. Cette dernière vérifie qu'un JSON GADM existe localement et retourne son chemin ; si non, elle s'arrête avec un message expliquant où télécharger le fichier. `sf::read_sf()` lit ensuite le JSON et retourne un objet `sf` (data.frame + géométrie).

```
source("../_commons/helpers/fetch_data.R")

# Charger les 3 niveaux administratifs GADM v4.1 telecharges manuellement
# dans datasets/cameroun/admin_boundaries/
adm0 <- read_sf(fetch_gadm_cameroon(0))
adm1 <- read_sf(fetch_gadm_cameroon(1))
adm3 <- read_sf(fetch_gadm_cameroon(3))
```

Premier réflexe : que contient l'objet ? `print()` sur un `sf` affiche un en-tête standardisé (type de géométrie, dimensions, emprise, CRS) puis les premières lignes du tableau attributaire. `names()` liste les colonnes, `nrow()` compte les entités, `st_geometry_type()` retourne le type géométrique de chaque entité (MULTIPOLYGON ici) — `table()` les compte par type.

```
print(adm1)           # affichage standardisé sf : géométries, bbox, CRS, attributs
```

Simple feature collection with 10 features and 11 fields

Geometry type: MULTIPOLYGON

Dimension: XY

Bounding box: xmin: 8.4995 ymin: 1.6523 xmax: 16.191 ymax: 13.0774

Geodetic CRS: WGS 84

A tibble: 10 × 12

	GID_1	GID_0	COUNTRY	NAME_1	VARNAME_1	NL_NAME_1	TYPE_1	ENGTYPE_1	CC_1	HASC_1
	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	CMR.1...	CMR	Camero...	Adama...	Adamoua ...	NA	Région	Region	NA	CM.AD
2	CMR.2...	CMR	Camero...	Centre	Central	NA	Région	Region	NA	CM.CE
3	CMR.3...	CMR	Camero...	Est	East	NA	Région	Region	NA	CM.ES
4	CMR.4...	CMR	Camero...	Extrê...	Extreme-...	NA	Région	Region	NA	CM.EN
5	CMR.5...	CMR	Camero...	Litto...	Litoral	NA	Région	Region	NA	CM.LT
6	CMR.7...	CMR	Camero...	Nord	North	NA	Région	Region	NA	CM.NO
7	CMR.6...	CMR	Camero...	Nord-...	North-We...	NA	Région	Region	NA	CM.NW
8	CMR.8...	CMR	Camero...	Ouest	West	NA	Région	Region	NA	CM.OU
9	CMR.1...	CMR	Camero...	Sud	South	NA	Région	Region	NA	CM.SU
10	CMR.9...	CMR	Camero...	Sud-O...	South-We...	NA	Région	Region	NA	CM.SW

i 2 more variables: ISO_1 <chr>, geometry <MULTIPOLYGON [°]>

```
names(adm1)
```

```
[1] "GID_1"      "GID_0"      "COUNTRY"    "NAME_1"     "VARNAME_1"  "NL_NAME_1"
[7] "TYPE_1"     "ENGTYPE_1"  "CC_1"       "HASC_1"     "ISO_1"      "geometry"
```

```
nrow(adm1)           # 10 régions attendues
```

```
[1] 10
```

```
st_geometry_type(adm1) |> table()
```

GEOMETRY	POINT	LINESTRING	POLYGON
0	0	0	0
MULTIPOINT	MULTILINESTRING	MULTIPOLYGON	GEOMETRYCOLLECTION
0	0	10	0
CIRCULARSTRING	COMPOUNDCURVE	CURVEPOLYGON	MULTICURVE
0	0	0	0
MULTISURFACE	CURVE	SURFACE	POLYHEDRALSURFACE
0	0	0	0
TIN	TRIANGLE		
0	0		

6.4 Anatomie d'un objet `sf`

Quatre fonctions à mémoriser pour interroger un objet `sf` : `class()` montre que c'est à la fois un `sf` ET un `data.frame` (héritage multiple) ; `st_bbox()` retourne l'emprise rectangulaire (xmin, ymin, xmax, ymax) ; `st_crs()$input` la description littérale du CRS ; `st_crs()$epsg` son code EPSG numérique. EPSG est un catalogue international de CRS — 4326 = WGS 84 géographiques par exemple.

```
class(adm1)           # "sf" + "data.frame"
```

```
[1] "sf"          "tbl_df"      "tbl"         "data.frame"
```

```
st_bbox(adm1)         # emprise géographique
```

```
  xmin  ymin  xmax  ymax
8.4995 1.6523 16.1910 13.0774
```

```
st_crs(adm1)$input    # quel CRS ?
```

```
[1] "WGS 84"
```

```
st_crs(adm1)$epsg     # code EPSG
```

```
[1] 4326
```

L'objet `adm1` est **à la fois** un `data.frame` (tu peux y appliquer `dplyr`) **et** un objet spatial (tu peux y appliquer des opérations géométriques). C'est le génie de `sf`.

Le bloc qui suit montre que `dplyr` marche directement sur un `sf`. `st_drop_geometry()` retire la colonne géométrie pour produire un tibble pur — utile quand on veut juste imprimer ou exporter une table d'attributs sans le poids des polygones.

```
# dplyr fonctionne directement
adm1 |>
  st_drop_geometry() |>      # enlever la géométrie pour table propre
  select(NAME_1, GID_1) |>
  arrange(NAME_1)
```

NAME_1	GID_1
<chr>	<chr>
Adamaoua	CMR.1_1
Centre	CMR.2_1
Est	CMR.3_1
Extrême-Nord	CMR.4_1
Littoral	CMR.5_1
Nord	CMR.7_1
Nord-Ouest	CMR.6_1
Ouest	CMR.8_1
Sud	CMR.10_1
Sud-Ouest	CMR.9_1

1-10 of 10 rows

6.5 Premier filtre spatial : la région du Centre

Filtrer un `sf`, c'est juste filtrer un `data.frame` — la géométrie suit automatiquement. On obtient un objet `sf` à une seule ligne. `st_geometry()` extrait juste la géométrie (sans les attributs), ce qui est utile pour un `plot()` minimaliste.

```
centre <- adm1 |> filter(NAME_1 == "Centre")
plot(st_geometry(centre), main = "Région du Centre")
```

Région du Centre



7 Quatrième session — Premier coup d’œil cartographique

🕒 ~1 heure. Trois moteurs : `plot()` (15 min), `ggplot` + `geom_sf` (25 min), `tmap` (20 min).

7.1 Avec `plot()` (base R)

Le plus minimaliste, utile pour vérifier rapidement. `plot(st_geometry(x))` dessine uniquement les contours sans coloration thématique. `add = TRUE` superpose la couche suivante sur le même graphique, ce qui permet d’empiler ici les régions et la frontière nationale ADM0.

```
plot(st_geometry(adm1), border = "grey50",  
     main = "Régions du Cameroun (GADM v4.1)")  
plot(st_geometry(adm0), border = "black", lwd = 2, add = TRUE)
```

Régions du Cameroun (GADM v4.1)

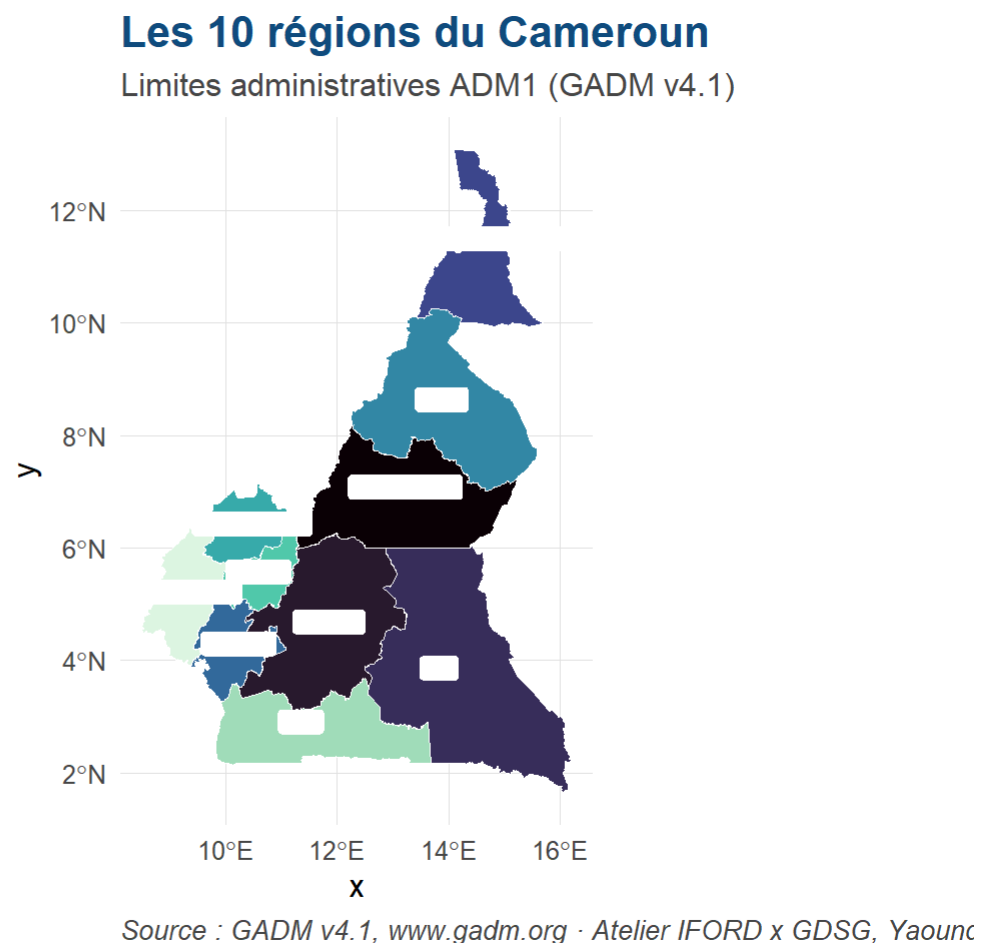


7.2 Avec `ggplot2`

`ggplot2` étendu par `sf` ajoute le `geom_sf()` qui sait dessiner n’importe quelle géométrie (point/ligne/polygone). On l’utilise comme n’importe quel `geom_*` : on mappe des attributs sur des esthétiques (`fill = NAME_1`). `geom_sf_label()` pose une étiquette au centroïde de chaque polygone. `scale_fill_viridis_d()` applique une palette discrète daltonien-friendly. `theme_iford()` et `iford_caption()` sont nos helpers maison (chargés via `source()` ci-dessous) pour homogénéiser le rendu.

```
source("../_commons/helpers/theme_iford.R")
library(ggplot2)

ggplot(adm1) +
  geom_sf(aes(fill = NAME_1), color = "white", linewidth = 0.3, show.legend = FALSE) +
  geom_sf_label(aes(label = NAME_1), size = 2.8, color = "white",
                fontface = "bold", label.padding = unit(0.1, "lines")) +
  scale_fill_viridis_d(option = "G") +
  labs(
    title = "Les 10 régions du Cameroun",
    subtitle = "Limites administratives ADM1 (GADM v4.1)",
    caption = iford_caption("Source : GADM v4.1, www.gadm.org")
  ) +
  theme_iford()
```



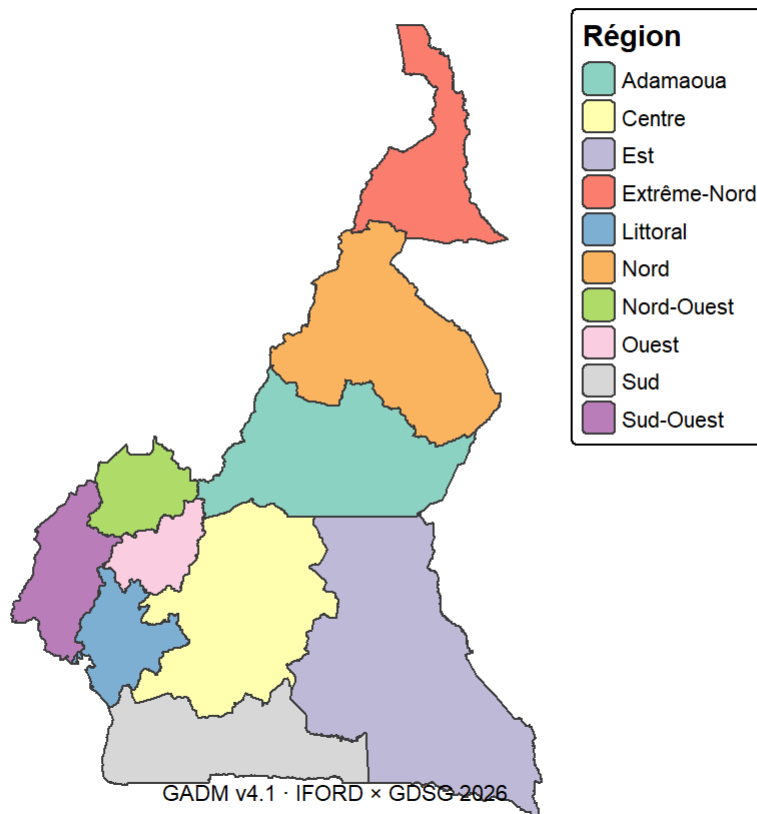
7.3 Avec **tmap** (cartographie thématique)

tmap est la grammaire « tidy » de la carte thématique (Hijmans 2024). Sa version 4 distingue deux **modes** : **plot** (statique, pour publication) et **view** (interactif Leaflet, pour exploration). Le bloc qui suit construit une carte par couches successives : **tm_shape()** fixe la couche de données, **tm_polygons()** dessine les polygones colorés par **NAME_1**, et **tm_title()** / **tm_credits()** posent les éléments d’habillage. **setup_tmap_iford()** est notre helper qui applique les défauts visuels de l’atelier.

```
library(tmap)
setup_tmap_iford()

tm_shape(adm1) +
  tm_polygons(
    fill = "NAME_1",
    fill.scale = tm_scale_categorical(values = "brewer.set3"),
    fill.legend = tm_legend("Région")
  ) +
  tm_title("Cameroun — découpage régional") +
  tm_credits("GADM v4.1 · IFORD × GDSG 2026", position = c("right", "bottom"))
```

Cameroun — découpage régional



Pour basculer en interactif on change de mode global avec `tmap_mode("view")`. Le bloc reste `eval: false` parce que le HTML statique ne sait pas afficher l’iframe Leaflet ici. `tmap_mode("plot")` revient au statique — important de toujours revenir, sinon les graphiques suivants seront eux aussi interactifs.

```
tmap_mode("view")
tm_shape(adm1) +
  tm_polygons(fill = "NAME_1", fill_alpha = 0.5)
tmap_mode("plot") # revenir au mode statique
```

8 Cinquième session — La pensée projective : un détour critique

🕒 ~1h30. Séance la plus dense de J1. **Pause de 5-10 min juste avant** est obligatoire.

8.1 Pourquoi un CRS ?

La Terre est (approximativement) un ellipsoïde. Une carte est plate. Il faut donc **projeter** : transformer des coordonnées sphériques (longitude, latitude) en coordonnées planes (x, y en mètres). Aucune projection n'est parfaite — chacune préserve une propriété (angles, aires, distances) au prix d'une autre. C'est un compromis politique autant que mathématique.

Trois familles à connaître :

Famille	Code EPSG	Usage
WGS 84 géographiques	4326	Standard d'échange (GPS, GeoJSON, KML). Coordonnées en degrés. Pas pour mesurer.
UTM	32632 (zone 32N) et 32633 (zone 33N) pour le Cameroun	Coordonnées planes locales en mètres. Calcul de distances et de surfaces.
Web Mercator	3857	Fonds OpenStreetMap, Google Maps. Déforme énormément les aires près des pôles — ne jamais l'utiliser pour des statistiques.

Le Cameroun chevauche deux fuseaux UTM (32N à l'ouest, 33N à l'est) ; pour les calculs nationaux, on utilise souvent une projection conique conforme adaptée.

8.2 Démonstration : la confusion qui coûte cher

On crée deux points isolés en WGS 84 (longitude, latitude) avec `st_sfc(st_point(...), crs = 4326)`. `st_sfc` = « simple feature collection » : une géométrie sans tableau d'attributs. On compare ensuite deux distances : une **naïve** (Pythagore sur les degrés, ce qui n'a aucun sens physique) et la **vraie** distance géodésique calculée par `st_distance()`. La différence pédagogique est saisissante.

```
yaounde <- st_sfc(st_point(c(11.5021, 3.8480)), crs = 4326)
douala  <- st_sfc(st_point(c(9.7679, 4.0511)), crs = 4326)

# Distance "naïve" sur des degrés (FAUX)
distance_fausse <- sqrt((11.5021 - 9.7679)^2 + (3.8480 - 4.0511)^2)
cat("Distance naïve sur degrés :", round(distance_fausse, 4), "(unité ??)\n")
```

Distance naïve sur degrés : 1.7461 (unité ??)

```
# Bonne distance grâce à sf (en mètres)
distance_correcte <- st_distance(yaounde, douala)
```

```
distance_correcte
```

```
Units: [m]
      [,1]
[1,] 193697.5
```

Sur le terrain Yaoundé–Douala fait ~240 km. Si on ne pose pas le CRS et qu'on traite les longitudes/latitudes comme des coordonnées planes, on obtient un nombre sans dimension impossible à interpréter.

8.3 Reprojecter un objet

`st_transform()` reprojette un objet `sf` vers un autre CRS en recalculant chaque coordonnée. Ici on passe de WGS 84 (degrés) à UTM 32N (mètres, EPSG:32632), ce qui est le minimum pour pouvoir mesurer des surfaces. `st_area()` retourne une superficie en unités du CRS (donc m²) — on divise par 10⁶ pour obtenir des km². Le résultat est ajouté comme une colonne au tibble pour pouvoir trier dessus.

```
adm1_utm32 <- st_transform(adm1, crs = 32632)
st_crs(adm1_utm32)$input
```

```
[1] "EPSG:32632"
```

```
# Surfaces en kilomètres carrés (jamais en degrés² !)
adm1_utm32$superficie_km2 <- as.numeric(st_area(adm1_utm32)) / 1e6
adm1_utm32 |>
  st_drop_geometry() |>
  select(NAME_1, superficie_km2) |>
  arrange(desc(superficie_km2))
```

NAME_1	superficie_km2
<chr>	<dbl>
Est	109782.84
Centre	68922.67
Nord	66802.86
Adamaoua	64266.74
Sud	47344.32
Extrême-Nord	34580.85
Sud-Ouest	24901.31
Littoral	20121.67
Nord-Ouest	17452.75
Ouest	13892.66
1-10 of 10 rows	

Vérification : la somme doit être proche de **475 442 km²** (superficie officielle du Cameroun). `sum()` agrège la colonne, `round()` arrondit à l'entier.

```
sum(adm1_utm32$superficie_km2) |> round()
```

L'écart résiduel (souvent $\pm 1\%$) vient de la généralisation des polygones GADM et de la déformation UTM hors fuseau. Pour des calculs « officiels » on utilisera des polygones BUCREP haute résolution avec une projection conforme nationale.

8.4 Démo critique : Web Mercator (EPSG:3857) ment sur les aires

Web Mercator est la projection des fonds OpenStreetMap, Google Maps, Mapbox. **Utile** pour les cartes web. **Inadaptée** aux statistiques car elle déforme les aires en s'éloignant de l'équateur. Vérifions sur le Cameroun.

On reprojette en EPSG:3857 et on recalcule les surfaces. On assemble ensuite un tibble qui met côte à côte la surface UTM (référence), la surface Web Mercator, et leur ratio. Si Web Mercator était fidèle, le ratio serait toujours 1.

```
adm1_webmerc <- st_transform(adm1, crs = 3857)
adm1_webmerc$webmerc_km2 <- as.numeric(st_area(adm1_webmerc)) / 1e6

comparaison <- tibble::tibble(
  region      = adm1_utm32$NAME_1,
  utm32N_km2  = round(adm1_utm32$superficie_km2),
  webmerc_km2 = round(adm1_webmerc$webmerc_km2),
  ratio_webmerc = round(adm1_webmerc$webmerc_km2 / adm1_utm32$superficie_km2, 2)
)
comparaison
```

region	utm32N_km2	webmerc_km2	ratio_webmerc
<chr>	<dbl>	<dbl>	<dbl>
Adamaoua	64267	65332	1.02
Centre	68923	69733	1.01
Est	109783	110183	1.00
Extrême-Nord	34581	35841	1.04
Littoral	20122	20377	1.01
Nord	66803	68282	1.02
Nord-Ouest	17453	17790	1.02
Ouest	13893	14114	1.02
Sud	47344	47700	1.01
Sud-Ouest	24901	25295	1.02

1-10 of 10 rows

Lecture critique : Web Mercator surestime la superficie d'un facteur **~1.0 au sud du Cameroun** (proche équateur) à **~1.04 au nord** (Maroua, $\sim 11^\circ$ latitude). Sur le Groenland ou la Sibérie le facteur dépasse 10. Une choroplèthe en Web Mercator donne une **lecture politique faussée** : les régions du nord paraissent plus grandes qu'elles ne le sont.

Règle absolue

Web Mercator (EPSG:3857) est interdit pour toute statistique spatiale. Réservé strictement aux **fonds de carte web** où

on dessine des points/lignes sur une tuile OpenStreetMap. Pour mesurer, agréger, comparer : UTM ou Lambert.

8.5 Quand on cartographie pour le web

Le bon réflexe pour une carte interactive : **garder les données en CRS adapté** (UTM, Lambert) pour les calculs, et **laisser leaflet / tmap view reprojeter en Web Mercator** automatiquement à l'affichage. Ne **jamais** stocker ses données projetées en Web Mercator.

Le bloc qui suit montre la bonne pratique : on garde `adm1_utm32` (UTM) comme source, et on active le mode `view` pour que `tmap` se charge de la reprojection web automatiquement, sans qu'on n'altère le fichier source.

```
tmap_mode("view")
tm_shape(adm1_utm32) + tm_polygons(fill = "NAME_1")
tmap_mode("plot")
```

9 Sixième session — Première vraie carte : densité régionale

 **~1 heure.** Récap appliqué : on combine TOUT ce qu'on a vu sur un seul livrable.

On combine les indicateurs simulés et la géométrie. D'abord on assemble une table population minimaliste avec `tibble()`. Ensuite `left_join()` la fusionne avec `adm1_utm32` en utilisant `NAME_1` comme clé — comme `adm1_utm32` est un `sf`, le résultat reste un `sf` (la géométrie suit). `mutate()` calcule la densité (habitants par km²). `st_drop_geometry()` puis `arrange()` produisent un classement attributaire pur, sans le poids des polygones.

```
# Table de population (notre vecteur du début)
pop_tbl <- tibble(NAME_1 = regions, pop_2019 = pop_est)

# Jointure attributaire
adm1_pop <- adm1_utm32 |>
  left_join(pop_tbl, by = "NAME_1") |>
  mutate(densite = pop_2019 / superficie_km2)

adm1_pop |>
  st_drop_geometry() |>
  select(NAME_1, pop_2019, superficie_km2, densite) |>
  arrange(desc(densite))
```

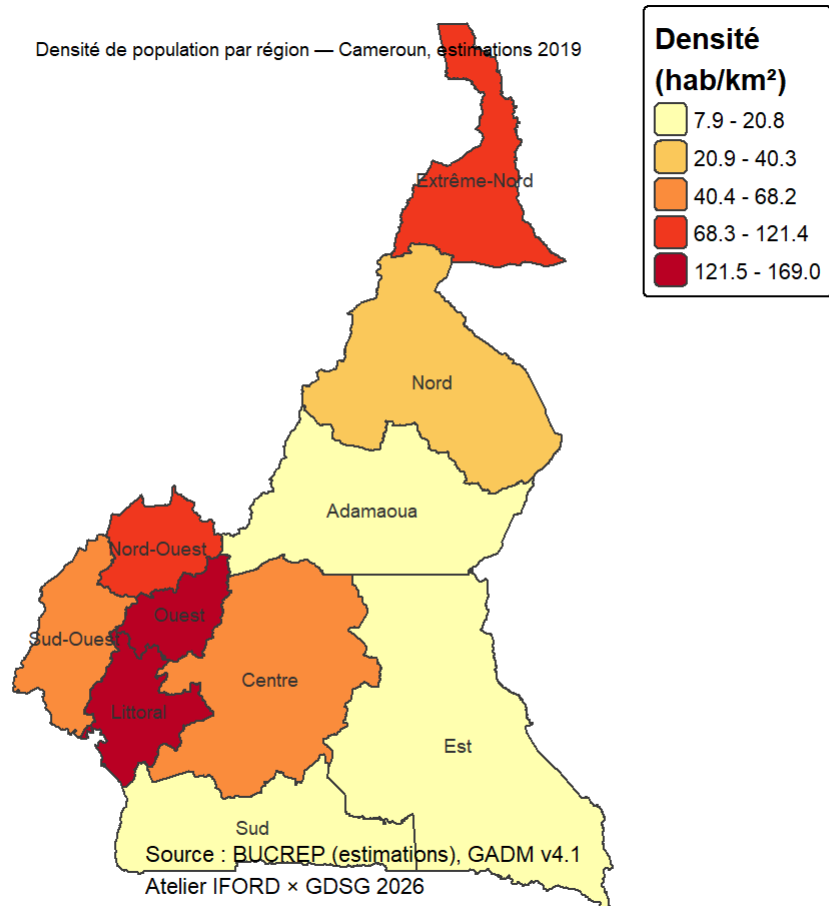
NAME_1	pop_2019	superficie_km2	densite
<chr>	<dbl>	<dbl>	<dbl>
Littoral	3400000	20121.67	168.972075
Ouest	2000000	13892.66	143.960935
Extrême-Nord	4200000	34580.85	121.454505
Nord-Ouest	2000000	17452.75	114.595120
Sud-Ouest	1700000	24901.31	68.269509

NAME_1	pop_2019	superficie_km2	densite
<chr>	<dbl>	<dbl>	<dbl>
Centre	4400000	68922.67	63.839663
Nord	2700000	66802.86	40.417429
Adamaoua	1340000	64266.74	20.850600
Sud	800000	47344.32	16.897486
Est	870000	109782.84	7.924736

1-10 of 10 rows

Pour la carte choroplèthe : `tm_scale_intervals(style = "jenks", n = 5)` découpe la variable continue en 5 classes optimisées (algorithme de Fisher-Jenks qui maximise la variance inter-classe). `values = "brewer.yl_or_rd"` applique une palette séquentielle jaune→rouge adaptée à une variable positive croissante. `tm_text("NAME_1")` pose le nom de chaque région au centroïde.

```
tm_shape(adm1_pop) +
  tm_polygons(
    fill = "densite",
    fill.scale = tm_scale_intervals(style = "jenks", n = 5,
                                    values = "brewer.yl_or_rd"),
    fill.legend = tm_legend("Densité\n(hab/km²)")
  ) +
  tm_text("NAME_1", size = 0.6, col = "grey20") +
  tm_title("Densité de population par région – Cameroun, estimations 2019",
           position = c("left", "top")) +
  tm_credits("Source : BUCREP (estimations), GADM v4.1\nAtelier IFORD × GDSG 2026",
            position = c("right", "bottom"))
```

Lecture critique en salle : le Littoral et l'Ouest sortent en tête. Pourquoi ? Que dirait un démographe qui voit ça ? Que disent les estimations BUCREP officielles versus WorldPop top-down pour 2020 ?

10 Septième session — Wrap-up et devoirs

🕒 ~30 minutes. Récap + diffusion de [exercice.qmd](#) pour le soir.

10.1 Ce qu'on a vu

- Vecteurs, types, coercion, recyclage.
- `data.frame` et `tibble`, indexation, `dplyr` (`filter`, `mutate`, `group_by`, `summarise`).
- Projet RStudio + [here](#) = reproductibilité.
- Lecture d'enquêtes démographiques (`haven` pour Stata).
- Anatomie d'un objet `sf` : `data.frame` + colonne géométrie + CRS.
- Trois moteurs de carte (`plot`, `ggplot2`, `tmap`).
- CRS : WGS 84 vs UTM vs Web Mercator, reprojection, calcul d'aires.
- Première carte choroplèthe propre.

10.2 Pour demain (J2)

Demain on attaque les opérations vectorielles sérieuses : `st_join`, `st_buffer`, `st_intersection`, `st_union`, dissolution de polygones (ADM3 → ADM1), et on entrera dans les fichiers BUCREP officiels du RGPH 4 si on les

a.

10.3 Devoir individuel (30 min, à faire le soir)

Le devoir individuel constitue l'exercice **Q6** de [exercice.qmd](#) ; le corrigé est dans [corrige.qmd](#). Résumé : créer un projet RStudio [tp_personnel](#), charger les ADM3 GADM Cameroun, filtrer une région d'origine, produire et exporter une carte des départements avec [tmap](#).

11 Sources et licences

Donnée	Source	Licence
GADM v4.1 ADM0-3 Cameroun	https://gadm.org/download_country.html	Académique gratuit
Estimations population régionale 2019	BUCREP (ordres de grandeur pédagogiques)	À actualiser RGPH 4
Microfichier EDS-MICS 2018 (réel ou simulé)	https://dhsprogram.com/	DHS Program

Code distribué sous **CC-BY 4.0** au nom du GDSG / IFORD.

11.1 Références méthodologiques

Pebesma ([2018](#)) ; Tatem ([2017](#)) ; Institut National de la Statistique du Cameroun and ICF ([2020](#)) ; Bureau Central des Recensements et des Études de Population (BUCREP) ([2026](#)).

Les références

Bureau Central des Recensements et des Études de Population (BUCREP). 2026. *Quatrième Recensement Général de la Population et de l'Habitat du Cameroun (RGPH 4)*. République du Cameroun.

Hijmans, Robert J. 2024. *terra: Spatial Data Analysis*.

Institut National de la Statistique du Cameroun and ICF. 2020. *Enquête Démographique et de Santé à Indicateurs Multiples du Cameroun 2018*. Yaoundé, Cameroun et Rockville, Maryland, USA.

Pebesma, Edzer. 2018. « Simple Features for R: Standardized Support for Spatial Vector Data ». *The R Journal* 10 (1): 439-46.

Tatem, Andrew J. 2017. « WorldPop, open data for spatial demography ». *Scientific Data* 4: 170004.
<https://doi.org/10.1038/sdata.2017.4>.